A Philosophy of Software Design

John Ousterhout

Chapter-by-chapter reading notes

How to read these notes

These notes follow the author's own structure: every chapter title and every section heading is reproduced word for word, in the book's order. The book has no Parts, so there are no Part overviews.

Text in "quotation marks" is the author's exact wording. Everything else is close paraphrase that keeps his terms, his steps of reasoning and his qualifiers. Nothing from outside the book has been added.

Blue box, "Place in the argument": why the chapter is here and why it has this title and this order of sections. **Gold box, "How this chapter builds the author's case"**: how it links to earlier and later chapters.

Bold terms are the author's defined terms. "Red flag" marks the author's own list of signs of unnecessary complexity.

Each chapter's notes are capped at one third of that chapter's length in characters (a 1:3 notes-to-book ceiling); most chapters come in well under it.

Contents

The Architecture of the Book at a Glance 3
Preface 4
1 Introduction (It's All About Complexity) 5
2 The Nature of Complexity 6
3 Working Code Isn't Enough (Strategic vs. Tactical Programming) 7
4 Modules Should Be Deep 8
5 Information Hiding (and Leakage) 9
6 General-Purpose Modules are Deeper 11
7 Different Layer, Different Abstraction 12
8 Pull Complexity Downwards 13
9 Better Together Or Better Apart? 14
10 Define Errors Out Of Existence 16
11 Design it Twice 18
12 Why Write Comments? The Four Excuses 19
13 Comments Should Describe Things that Aren't Obvious from the Code 20
14 Choosing Names 22
15 Write The Comments First (Use Comments As Part Of The Design Process) 23
16 Modifying Existing Code 24
17 Consistency 25
18 Code Should be Obvious 26
19 Software Trends 27
20 Designing for Performance 28
21 Conclusion 29
Parts of the Book Not Summarised 30

THE WHOLE BOOK

The Architecture of the Book at a Glance

The author states the book's purpose in the first sentence of Chapter 2: "This book is about how to design software systems to minimize their complexity." His last chapter repeats it: "This book is about one thing: complexity." The table gives, for each unit, his own words about what it is for.

Unit	Title	In the author's words
Preface	Preface	"the core problem of software design is still largely untouched"
Ch. 1	Introduction (It's All About Complexity)	"the greatest limitation in writing software is our ability to understand the systems we are creating"
Ch. 2	The Nature of Complexity	"The first step is to understand the enemy".
Ch. 3	Working Code Isn't Enough (Strategic vs. Tactical Programming)	"you must take a more strategic approach where you invest time to produce clean designs and fix problems"
Ch. 4	Modules Should Be Deep	"design systems so that developers only need to face a small fraction of the overall complexity at any given time"
Ch. 5	Information Hiding (and Leakage)	"each module should encapsulate a few pieces of knowledge, which represent design decisions"
Ch. 6	General-Purpose Modules are Deeper	"the module's functionality should reflect your current needs, but its interface should not"
Ch. 7	Different Layer, Different Abstraction	"each layer provides a different abstraction from the layers above and below it"
Ch. 8	Pull Complexity Downwards	"it is more important for a module to have a simple interface than a simple implementation"
Ch. 9	Better Together Or Better Apart?	"should they be implemented together in the same place, or should their implementations be separated?"
Ch. 10	Define Errors Out Of Existence	"reduce the number of places where exceptions must be handled"
Ch. 11	Design it Twice	"consider multiple options for each major design decision: design it twice"
Ch. 12	Why Write Comments? The Four Excuses	"without comments, you can't hide complexity"
Ch. 13	Comments Should Describe Things that Aren't Obvious from the Code	"comments should describe things that aren't obvious from the code"
Ch. 14	Choosing Names	"Good names are a form of documentation"
Ch. 15	Write The Comments First (Use Comments As Part Of The Design Process)	"The best time to write comments is at the beginning of the process"
Ch. 16	Modifying Existing Code	"keep complexity from creeping in as the system evolves"
Ch. 17	Consistency	"similar things are done in similar ways, and dissimilar things are done in different ways"
Ch. 18	Code Should be Obvious	"someone can read the code quickly, without much thought, and their first guesses about the behavior or meaning of the code will be correct"
Ch. 19	Software Trends	"challenge it from the standpoint of complexity"
Ch. 20	Designing for Performance	"clean design and high performance are compatible"
Ch. 21	Conclusion	"This book is about one thing: complexity".

Preface

Place in the argument — why this chapter, why this title

The Preface comes first because it states the problem the whole book answers, before any principle. It says why a book on design is needed and why the author is qualified to offer an opinion, and it asks you to test every idea against your own experience.

- The gap: “there has been surprisingly little conversation about how to design those programs or what good programs should look like.” Processes, tools, techniques and patterns are valuable, “but the core problem of software design is still largely untouched.” Parnas’s 1971 paper is the last big step.
- The core task: “The most fundamental problem in computer science is problem decomposition”, yet “We teach for loops and object-oriented programming, but not software design.”
- Can it be taught? Great programmers vary hugely in quality and productivity but “had difficulty articulating specific techniques”; evidence suggests performance relates “more to high-quality practice than innate ability.”
- The method: the course CS 190 at Stanford, taught like an English writing class: write a draft, get feedback, rewrite. The book’s principles “emerged from the class.”
- The source of authority is experience, not certainty: “To be honest, I don’t.” About 250,000 lines of code; “every problem described here is one that I have experienced personally, and every suggested technique is one that I have used successfully.”
- The stance: “This book is an opinion piece”; “take the suggestions in this book with a grain of salt. The overall goal is to reduce complexity; this is more important than any particular principle.”

How this chapter builds the author’s case

Sets the book’s two anchors: complexity reduction as the test of any idea, and learning by writing, reviewing and revising. Chapter 1 turns this into the thesis.

CHAPTER 1

Introduction (It's All About Complexity)

Place in the argument — why this chapter, why this title

Chapter 1 states the thesis: the limit on software is our ability to understand it, so design is about fighting complexity. The title says everything later is measured against complexity. The opening section frames the argument and 1.1 tells you how to use the book.

Chapter opening — untitled section before the first sub-heading in the book

- Programming is "one of the purest creative activities", so "the greatest limitation in writing software is our ability to understand the systems we are creating." Complexity "increases inevitably"; tools help, "but there is a limit to what we can do with tools alone."
- Two attacks on complexity: make code "simpler and more obvious" (e.g. remove special cases, use identifiers consistently), or "encapsulate it" by modular design, so one module can be worked on without understanding the others.
- Design is continuous: the waterfall model "rarely works well for software" because you cannot foresee the problems; the incremental (agile) approach works because software is malleable. "Software design is never done."
- Two goals: describe the nature of complexity, and (the harder one) present techniques to minimize it. There is no recipe, only concepts that "border on the philosophical" such as "classes should be deep" and "define errors out of existence."

1.1 How to use this book

- Principles are abstract, so use the book with code reviews; look for **red flags**, "signs that a piece of code is probably more complicated than it needs to be", and try alternatives until the flag disappears.
- Use moderation: "Every rule has its exceptions"; sections called "Taking it too far" mark the limits. Examples are in Java or C++ but the ideas apply to functions and to larger modules.

How this chapter builds the author's case

Chapter 2 defines and diagnoses complexity; chapter 3 sets the working attitude (strategic); chapters 4 to 10 give the modular-design and simplification techniques; later chapters apply them to comments, names, consistency and performance.

CHAPTER 2

The Nature of Complexity

Place in the argument — why this chapter, why this title

"The first step is to understand the enemy." The chapter comes straight after the Introduction's claim that complexity is the problem, and it makes the claim usable: a definition (2.1), how complexity shows up (2.2), what produces it (2.3), how it grows (2.4). The order runs from what it is, to how you notice it, to why it exists, to why it is hard to stop.

Chapter opening — untitled section before the first sub-heading in the book

- Recognizing complexity is "a crucial design skill": it lets you "make good choices among alternatives." The chapter also "lays out some basic assumptions" that later chapters "take the material of this chapter as given."

2.1 Complexity defined

- Definition: "Complexity is anything related to the structure of a software system that makes it hard to understand and modify the system." Simple means easy to understand and modify; in cost terms, a complex system makes even small improvements take a lot of work.
- It is what a developer experiences "at a particular point in time when trying to achieve a particular goal", not system size.
- It is weighted by use: overall complexity is the complexity of each part weighted by the fraction of time developers spend on it, so "Isolating complexity in a place where it will never be seen is almost as good as eliminating the complexity entirely."
- It is judged by the reader: "If you write a piece of code and it seems simple to you, but other people think it is complex, then it is complex."

2.2 Symptoms of complexity

- **Change amplification**: a seemingly simple change needs modifications in many places (banner colour set on every page versus once centrally).
- **Cognitive load**: how much a developer needs to know to finish a task (e.g. a C function that returns memory the caller must free). Fewer lines of code is not simpler if it raises cognitive load.
- **Unknown unknowns**: it is not obvious which code must change or what you must know (a darker emphasis colour hard-coded on some pages). "Of the three manifestations of complexity, unknown unknowns are the worst."
- The opposite goal is an **obvious** system, where "a developer can make a quick guess about what to do, without thinking very hard, and yet be confident that the guess is correct."

2.3 Causes of complexity

- "Complexity is caused by two things: dependencies and obscurity."
- A dependency exists "when a given piece of code cannot be understood and modified in isolation." They cannot be eliminated, so aim to reduce their number and make the rest "as simple and obvious as possible" (the central banner variable replaced a hidden dependency with an obvious one).
- Obscurity "occurs when important information is not obvious": generic names, undocumented units, hidden tables, inconsistency. Needing extensive documentation "is often a red flag that the design isn't quite right."
- Mapping: dependencies lead to change amplification and cognitive load; obscurity creates unknown unknowns and adds to cognitive load.

2.4 Complexity is incremental

- "Complexity isn't caused by a single catastrophic error; it accumulates in lots of small chunks." Once it has accumulated, fixing one item makes little difference, so adopt a "zero tolerance" philosophy.

2.5 Conclusion

- Dependencies and obscurities accumulate into the three symptoms; "complexity makes it difficult and risky to modify an existing code base."

How this chapter builds the author's case

Builds on the Introduction's goal by giving the two targets (dependencies, obscurity) and three symptoms that every later chapter uses. The "zero tolerance" point leads to chapter 3; the obvious-code goal returns in chapter 18; obscurity returns in the comments and naming chapters.

CHAPTER 3

Working Code Isn't Enough (Strategic vs. Tactical Programming)

Place in the argument — why this chapter, why this title

Chapter 2 said complexity grows in small steps; chapter 3 asks who adds those steps and why, and answers with a mindset. The title gives the thesis, the subtitle names the two mindsets. It comes before any technique because the techniques only work if you invest the time.

Chapter opening — untitled section before the first sub-heading in the book

- "One of the most important elements of good software design is the mindset you adopt." Organizations encourage a tactical mindset, but the strategic approach "is actually cheaper than the tactical approach over the long run."

3.1 Tactical programming

- In tactical programming "your main focus is to get something working", which makes "it nearly impossible to produce a good system design." Each shortcut seems a reasonable compromise, but they accumulate and each patch creates "more complexity, which then requires more patches."
- The **tactical tornado**: a prolific programmer who "pumps out code far faster than others" but leaves "a wake of destruction"; the engineers who clean up look slower.

3.2 Strategic programming

- "Working code isn't enough." The primary goal "must be to produce a great design, which also happens to work", because most code is written by extending the existing code base.
- It "requires an investment mindset": proactive investments (try alternative designs, good documentation) and reactive ones (when you find a design problem, fix it rather than patch around it).

3.3 How much to invest?

- Not a huge up-front design (that is waterfall). "Make lots of small investments on a continual basis", about 10-20% of development time. Early projects take 10-20% longer, then investments become free as past investment saves more than future investment costs.
- Tactical gives the reverse: faster at first, then at least 10-20% slower. The author notes the curves are qualitative only.

3.4 Startups and investment

- Startups feel pressure to be tactical, but spaghetti code "is nearly impossible to fix", and the best engineers "care deeply about good design", so a wrecked code base hurts recruiting.
- Facebook's "Move fast and break things" became "Move fast with solid infrastructure"; Google and VMware succeeded with a strategic approach. A company can succeed either way, but "it's a lot more fun to work in a company that cares about software design."

3.5 Conclusion

- "Good design doesn't come for free." Invest "today, not tomorrow", since after one crunch comes another; make "continuous small investments in good design."

How this chapter builds the author's case

Turns chapter 2's incremental complexity into a rule of conduct (zero tolerance, continual small investment). It frames chapters 4 to 10 as the techniques to invest in, and comes back in chapter 16 (stay strategic when modifying code) and chapter 19 (agile and test-driven development as tactical risks).

CHAPTER 4

Modules Should Be Deep

Place in the argument — why this chapter, why this title

Chapter 2 named dependencies and obscurity as the causes of complexity; chapter 4 begins the answer: modular design, so a developer faces only "a small fraction of the overall complexity at any given time." The title is the rule. The sections move from modules, to interfaces, to abstractions, to depth, to its opposite (shallow modules, classitis), to Java and Unix examples.

Chapter opening — untitled section before the first sub-heading in the book

- Modular design is "one of the most important techniques for managing software complexity."

4.1 Modular design

- A system is decomposed into relatively independent modules; the goal "is to minimize the dependencies between modules." Each module has an **interface** (what a developer in another module must know) and an **implementation**.
- A module is "any unit of code that has an interface and an implementation" (classes, methods, subsystems, services).
- "The best modules are those whose interfaces are much simpler than their implementations": a simple interface limits the complexity imposed on others, and changes behind it affect no one.

4.2 What's in an interface?

- Formal parts (signatures) are checked by the language. Informal parts (behaviour, usage constraints) can only be described in comments, and "are larger and more complex than the formal aspects."

4.3 Abstractions

- An abstraction is "a simplified view of an entity, which omits unimportant details." It fails in two ways: including unimportant details (more cognitive load) or omitting important ones, a **false abstraction**, which creates obscurity. The file-system example: flushing rules must be visible.

4.4 Deep modules

- "The best modules are those that provide powerful functionality yet have simple interfaces." Cost is the interface, benefit is the functionality; "more, or larger, interfaces are not necessarily better!"
- Examples: Unix file I/O (five basic calls hiding hundreds of thousands of lines) and the garbage collector, which "has no interface at all."

4.5 Shallow modules

- A shallow module has an interface that is complex relative to its functionality; a linked-list class is an example, and a one-line method that only calls data.put(attribute, null) "adds complexity … but provides no compensating benefit."
- **Red flag: shallow module.** "Small modules tend to be shallow."

4.6 Classitis

- Conventional wisdom says classes should be small; the extreme is **classitis**, from the view that "classes are good, so more classes are better." The accumulated interfaces create "tremendous complexity at the system level."

4.7 Examples: Java and Unix I/O

- Java needs three objects to read serialized objects from a file, and buffering must be requested explicitly. Interfaces "should be designed to make the common case as simple as possible"; Unix made sequential I/O the default. The effective complexity of an interface is that of its commonly used features.

4.8 Conclusion

- "The most important issue in designing classes and other modules is to make them deep."

How this chapter builds the author's case

Chapter 4 supplies the central test (depth) used from here on. Chapter 5 shows what depth depends on (information hiding), chapter 6 how generality produces it, chapter 7 how layers keep it, chapter 8 which side of an interface complexity should sit on. The classitis point returns in chapter 9 (splitting and joining).

CHAPTER 5

Information Hiding (and Leakage)

Place in the argument — why this chapter, why this title

Chapter 4 argued that modules should be deep; chapter 5 gives "the most important technique for achieving deep modules." The title pairs the technique with its failure, so the sections go: the idea (5.1), its opposite (5.2), a common cause of the opposite (5.3), then an HTTP case study from student projects (5.4-5.7), then application inside a class (5.8) and its limit (5.9).

Chapter opening — untitled section before the first sub-heading in the book

- Chapter 5 and the next few "discuss techniques for creating deep modules."

5.1 Information hiding

- "Each module should encapsulate a few pieces of knowledge, which represent design decisions." The knowledge is in the implementation and not in the interface.
- It reduces complexity twice: it simplifies the interface (less cognitive load), and it makes the system easier to evolve, since a change to hidden information affects only one module (a new TCP congestion mechanism needs no change in code that uses TCP).
- Private is not the same thing: getters and setters can expose private variables "just as exposed as if the variables were public." Partial hiding also has value.

5.2 Information leakage

- Leakage "occurs when a design decision is reflected in multiple modules." Back-door leakage (two classes that both know a file format) "is more pernicious than leakage through an interface, because it isn't obvious."
- Remedies: reorganize so the knowledge affects one class, merge the classes, or extract a new class with a simple interface; otherwise you have only replaced back-door leakage with leakage through an interface.
- **Red flag: information leakage.**

5.3 Temporal decomposition

- Structuring modules by time order (read, modify, write) makes reading and writing both know the file format. The fix is one class for both. "Focus on the knowledge that's needed to perform each task, not the order in which tasks occur."
- **Red flag: temporal decomposition.**

5.4 Example: HTTP server

- Students implemented classes so a web server can receive HTTP requests and send responses; the examples show what they did well and badly.

5.5 Example: too many classes

- One team used a class to read the request into a string and another to parse it, but the headers must be parsed to know the body length, so both needed the request structure and callers had to call two methods in order. Merging them hides the format and gives one method.
- General theme: "information hiding can often be improved by making a class slightly larger", by bringing together everything for a capability or by raising the level of the interface. Not to the extreme of one class for everything.

5.6 Example: HTTP parameter handling

- Good choices: hiding whether a parameter is in the first line or the body, and hiding URL encoding by decoding values.
- A shallow mistake: a method returning the internal Map exposes the representation, adds work for callers, and risks callers modifying it. Better: getParameter and getIntParameter, which hide the representation and the string-to-integer conversion.

5.7 Example: defaults in HTTP responses

- The protocol version and Date header can be defaulted by the library. Defaults show that "interfaces should be designed to make the common case as simple as possible" and are partial information hiding. "The best features are the ones you get without even knowing they exist."
- **Red flag: overexposure** (a common feature's API forces users to learn about rarely used ones).

5.8 Information hiding within a class

- Make each private method encapsulate something, and minimize the places where each instance variable is used, to remove dependencies inside the class.

5.9 Taking it too far

- Hiding is only right when the information “is not needed outside its module.” Performance-affecting configuration may have to be exposed; better still if the module adjusts itself.

5.10 Conclusion

- Hiding more information tends to add functionality and shrink the interface (deeper); the reverse gives shallow modules. Design around pieces of knowledge, not execution order.

How this chapter builds the author’s case

Gives chapter 4’s depth a mechanism and a diagnostic (leakage). Chapter 6 shows generality helps hiding, chapter 9 returns to when to merge or split classes, and chapter 8 and 10 carry “common case simple” and defaults further.

CHAPTER 6

General-Purpose Modules are Deeper

Place in the argument — why this chapter, why this title

Chapter 5 said hiding information deepens modules; chapter 6 takes one design decision that most affects it: general-purpose or special-purpose? The title states the author's answer. The opening sets out both sides, 6.1 gives the stance, 6.2-6.4 test it on a text-editor class, 6.5 gives questions for finding the balance.

Chapter opening — untitled section before the first sub-heading in the book

- General-purpose fits the investment mindset but may add unneeded facilities; special-purpose fits incremental development ("you can always refactor it to make it general-purpose") but the future is hard to predict.

6.1 Make classes somewhat general-purpose

- "The sweet spot is to implement new modules in a somewhat general-purpose fashion": the functionality reflects current needs, "but its interface should not." Do not "get carried away."
- The most important and perhaps surprising benefit: "simpler and deeper interfaces", beneficial even if the module is used for only its original purpose.

6.2 Example: storing text for an editor

- Student teams built a text class for a GUI editor with special-purpose methods such as backspace, delete and deleteSelection(Selection), shaped by the user interface.
- Result: many shallow methods, often used in one place; high cognitive load for both sides; and information leakage, since UI ideas (selection, backspace key) appeared in the text class and each new UI operation needed a new text method.

6.3 A more general-purpose API

- Only two methods for modification: insert(position, newText) and delete(start, end), with a generic Position and changePosition. Backspace and delete become one line each in the UI code.
- The UI code is "a bit longer" but "more obvious", and there is less code overall. The class could also serve other applications (e.g. replace-all, adding only findNext).

6.4 Generality leads to better information hiding

- Cleaner separation: the text class does not know the UI, and new UI features need no new text methods.
- The old backspace method "was a false abstraction": the UI developer needs to know which characters it deletes. "One of the most important elements of software design is determining who needs to know what, and when." Hiding that detail "just creates obscurity."

6.5 Questions to ask yourself

- "What is the simplest interface that will cover all my current needs?" (fewer methods, but not by adding many arguments).
- "In how many situations will this method be used?" A method for one use is a red flag.
- "Is this API easy to use for my current needs?" Single-character insert and delete is general but forces loops and is inefficient, so support ranges.

6.6 Conclusion

- General-purpose interfaces "tend to be simpler, with fewer methods that are deeper" and separate classes better. "Making your modules somewhat general-purpose is one of the best ways to reduce overall system complexity."

How this chapter builds the author's case

Applies chapter 5's information hiding to a design choice and gives a second route to depth. It answers chapter 3's investment question for interfaces. Chapter 7 asks how layers should differ, and chapter 9 returns to separating general-purpose from special-purpose code.

CHAPTER 7

Different Layer, Different Abstraction

Place in the argument — why this chapter, why this title

Chapters 4 to 6 were about single modules; chapter 7 looks at how modules stack. The title is the rule, with the red flag in the opening: adjacent layers with similar abstractions. The sections follow the forms the problem takes: pass-through methods, decorators, interface versus implementation, pass-through variables, with a section on when duplication is fine.

Chapter opening — untitled section before the first sub-heading in the book

- Systems are layered, and "each layer provides a different abstraction from the layers above and below it" (a file system: files, cached blocks, device drivers; TCP: reliable byte stream over best-effort packets). "If a system contains adjacent layers with similar abstractions, this is a red flag."

7.1 Pass-through methods

- A pass-through method "does little except invoke another method, whose signature is similar or identical." In a student editor, 13 of the 15 public methods of TextDocument were pass-through. **Red flag: pass-through method.**
- They make classes shallower, add dependencies, and signal confusion over responsibilities: "the interface to a piece of functionality should be in the same class that implements the functionality."
- Fixes: expose the lower class directly, redistribute the functionality, or merge the classes (the student went from three classes to two).

7.2 When is interface duplication OK?

- Same signature is fine if each method adds "significant functionality": a **dispatcher** (a web server choosing a handler from the URL), or several implementations of one interface (disk drivers), which lowers cognitive load.

7.3 Decorators

- A decorator offers an API like the object it wraps (BufferedInputStream; a ScrollableWindow). It separates special extensions from a generic core, but decorators "tend to be shallow" and can cause "an explosion of shallow classes."
- Alternatives: add the functionality to the underlying class (buffering should have been combined with InputStream), merge it with its use case or with an existing decorator, or make it a stand-alone class (scrollbars). "Sometimes decorators make sense, but there is usually a better alternative."

7.4 Interface versus implementation

- A class's interface should differ from its implementation. A text class with a line-oriented API (getLine, putLine) forced callers to split and join lines. A character-oriented insert and delete over line-based storage hides that work, and "the difference represents valuable functionality."

7.5 Pass-through variables

- A variable passed down a long chain of methods that do not use it (a certificate setting needed only by method m3) forces intermediate methods to know about it.
- Options: a shared object (which may itself be pass-through); a global variable (blocks two independent instances, e.g. in tests); or a **context object** holding global state, with a reference saved in major objects and passed explicitly only to constructors. Contexts are "far from an ideal solution" (grab-bag, thread safety; keep variables immutable), but "I haven't found a better solution."

7.6 Conclusion

- Each element of design infrastructure "adds complexity", so it must eliminate more than it adds. Same-abstraction layers, pass-through methods and arguments do not.

How this chapter builds the author's case

Extends depth from one module to the layering of the system. It uses chapter 4's shallow-module red flag and the Java I/O example, and chapter 6's text-editor example. Chapter 8 then asks on which side of an interface complexity should live, and chapter 9 when to merge or split.

CHAPTER 8

Pull Complexity Downwards

Place in the argument — why this chapter, why this title

Chapters 6 and 7 dealt with how many layers and how general; chapter 8 asks who should bear unavoidable complexity. The title is the instruction. It is another route to deep classes, so it follows the layers chapter. Two examples (text class, configuration parameters) show it, then 8.3 gives its limit.

Chapter opening — untitled section before the first sub-heading in the book

- Should users or the module handle unavoidable complexity? "Most modules have more users than developers, so it is better for the developers to suffer than the users." "It is more important for a module to have a simple interface than a simple implementation."
- The tempting opposite, throwing exceptions or adding configuration parameters, means many people deal with a problem instead of one.

8.1 Example: editor text class

- A line-oriented text interface was simple to implement but forced the UI to split and join lines. A character-oriented interface "pulls complexity downward": the text class does the splitting, and overall complexity falls.

8.2 Example: configuration parameters

- Parameters move complexity up. Advocates say users know their domains, but parameters are "an easy excuse to avoid dealing with important issues"; values are hard to choose and go out of date. A protocol can measure response times and compute its own retry interval.
- Ask: "will users (or higher-level modules) be able to determine a better value than we can determine here?" Compute defaults automatically. "Ideally, each module should solve a problem completely."

8.3 Taking it too far

- Pull down only if the complexity (a) is closely related to the class's functionality, (b) simplifies much elsewhere, and (c) simplifies the interface. Putting backspace in the text class "just resulted in information leakage."

8.4 Conclusion

- "Take a little bit of extra suffering upon yourself in order to reduce the suffering of your users."

How this chapter builds the author's case

Reuses chapter 6's text-editor example for the opposite lesson from the UI-shaped methods: pull down what belongs to the class, not what leaks the UI. Anticipates chapter 10 (exceptions are also complexity pushed to callers) and chapter 9's question of where code belongs.

CHAPTER 9

Better Together Or Better Apart?

Place in the argument — why this chapter, why this title

Chapters 4 to 8 argued for deep modules and warned against splitting too much; chapter 9 asks the decision directly: "given two pieces of functionality, should they be implemented together in the same place, or should their implementations be separated?" The title is that question. The sections give four rules (9.1-9.4), then three worked cases (9.5-9.7), then methods (9.8).

Chapter opening — untitled section before the first sub-heading in the book

- The goal is to "reduce the complexity of the system as a whole." Small components are not automatically simpler, since subdividing creates more components and interfaces, more code to manage them, **separation** (bad if components depend on each other) and duplication.
- Signs that code is related: it shares information; it is used together (only compelling if bidirectional, so a hash table and a block cache stay separate); it overlaps conceptually; or one piece is hard to understand without the other.

9.1 Bring together if information is shared

- The HTTP reader could not find the end of a request without parsing headers, so reading and parsing shared knowledge of the format; "when the two classes were combined into one, the code got shorter and simpler."

9.2 Bring together if it will simplify the interface

- Combining can remove the interface between the parts (the string passed from reader to parser) and allow automatic behaviour, such as buffering by default in Java's FileInputStream.

9.3 Bring together to eliminate duplication

- Factor repeated code into a method if the snippet is long and the signature simple; or restructure so it runs in one place (a goto to shared cleanup code at the end of a method is acceptable here, though "generally considered a bad idea").
- **Red flag: repetition**: "you haven't found the right abstractions."

9.4 Separate general-purpose and special-purpose code

- A module with a general-purpose mechanism should not include code specialized for one use. Lower layers tend to be general, upper layers special, so "pull the special-purpose code upwards." The text class keeps text operations and the UI keeps deleting the selection.
- **Red flag: special-general mixture**, which causes information leakage.

9.5 Example: insertion cursor and selection

- One team combined the cursor and selection in one object, but higher-level code still treated them separately and the combined object needed a boolean for which end was the cursor. "The selection and cursor were not closely enough related to combine them." Two Positions for the selection and one for the cursor made both usage and implementation simpler.

9.6 Example: separate class for logging

- A separate NetworkErrorLogger class had one-line methods needing documentation, each called from one place, and readers flipped between call and method. "It would be better to eliminate the logging methods and place the logging statements at the locations where the errors were detected."

9.7 Example: editor undo mechanism

- Undo built into the text class mixed a general list-of-actions mechanism with special handlers for the text, selection and cursor, leaking between text class and UI.
- The fix is a separate History class that knows nothing about the actions: it holds Action objects (undo and redo) and groups them with fences. Three parts: the general mechanism (History), specific actions (classes that understand them), and the grouping policy (high-level UI code). "The key design decision was the one that separated the general-purpose part of the undo mechanism from the special-purpose parts."
- Note: special-purpose undo code for text belongs in the text class, since it is related to other text functions.

9.8 Splitting and joining methods

- “Length by itself is rarely a good reason for splitting up a method.” Splitting adds interfaces and separates related code. Methods of hundreds of lines “are fine if they have a simple signature and are easy to read.”
- Goal: “Each method should do one thing and do it completely”, and be deep. Split only for cleaner abstractions: (1) extract a cleanly separable subtask, typically general-purpose; or (2) divide a method with an overly complex interface into parts, where most callers need only one. Splitting so callers invoke both parts risks shallow methods.
- Joining can replace shallow methods with a deeper one, remove duplication or dependencies, or simplify the interface.
- **Red flag: conjoined methods.** “It should be possible to understand each method independently.”

9.9 Conclusion

- “Pick the structure that results in the best information hiding, the fewest dependencies, and the deepest interfaces.”

How this chapter builds the author’s case

Draws together chapters 4 to 8: classitis and shallow modules (4), leakage and merging reader and parser (5), general versus special purpose (6) and the editor examples (6 and 8). It supplies the decision rule for splitting that chapters 4 and 5 only hinted at.

CHAPTER 10

Define Errors Out Of Existence

Place in the argument — why this chapter, why this title

Chapter 8 said to take suffering on yourself rather than push it to users; exceptions are the biggest case. The title is the chapter's lead technique. It first shows why exceptions are costly (10.1-10.2), then four ways to reduce the places they must be handled: define out (10.3-10.5), mask (10.6), aggregate (10.7), crash (10.8). 10.9 extends the idea to special cases, 10.10 sets the limit.

Chapter opening — untitled section before the first sub-heading in the book

- "Exception handling is one of the worst sources of complexity in software systems." The key lesson is "to reduce the number of places where exceptions must be handled", by changing semantics "so that the normal behavior handles all situations."

10.1 Why exceptions add complexity

- An exception is "any uncommon condition that alters the normal flow of control", including special return values. Handling is harder than normal code: you either move forward despite it or abort and restore consistent state.
- Handling creates more exceptions (a resent packet may arrive twice), language support is "verbose and clunky" (the try-catch boilerplate outweighs the normal code), and it is hard to test: "code that hasn't been executed doesn't work." A study found over 90% of catastrophic failures in distributed data-intensive systems were caused by incorrect error handling.

10.2 Too many exceptions

- Programmers think "the more errors detected, the better", producing an over-defensive style. The author's own mistake: Tcl's unset threw an error for a missing variable, though it is mostly used for cleanup; "one of the biggest mistakes I made in the design of Tcl."
- Exceptions are part of the interface, so more exceptions mean shallower classes. "Throwing exceptions is easy; handling them is hard."

10.3 Define errors out of existence

- "Define your APIs so that there are no exceptions to handle." Redefine unset to ensure a variable no longer exists; if it is already gone, "its work is already done."

10.4 Example: file deletion in Windows

- Windows refuses to delete an open file. Unix removes the name, returns success, and frees the data when the last process closes it. This defines away errors for the deleter and for the processes using the file.

10.5 Example: Java substring method

- substring throws IndexOutOfBoundsException, turning a one-line call into 5-10 lines of range checks. Defining it as "returns the characters of the string (if any) with index greater than or equal to beginIndex and less than endIndex" removes the exception and makes the method deeper.
- Objection that errors catch bugs: the error-ful approach adds code and so other bugs. "The best way to reduce bugs is to make software simpler."

10.6 Mask exceptions

- Handle the condition at a low level so higher levels do not know about it: TCP resends lost packets; NFS clients retry forever and the application hangs ("not responding still trying"), better than every application aborting and collapsing the user's environment.
- Masking is "an example of pulling complexity downward", and gives deeper classes.

10.7 Exception aggregation

- Handle many exceptions with one handler. In a web server, handlers around each getParameter call are all alike; let NoSuchParameter propagate to the top-level dispatcher, with the message carried in the exception. New methods that throw the same way plug in unchanged.
- Pattern: an exception that aborts the current request, cleans up and continues with the next, caught once near the top of the request loop, distinct from fatal ones.
- Aggregation works best when the exception propagates up; masking when it is handled low. RAMCloud "promotes" a corrupted object to a server crash so it reuses crash recovery and exercises

it more, which only suits rare errors. It replaces several special-purpose mechanisms with one general-purpose mechanism.

10.8 Just crash?

- For errors “not worth trying to handle”, print diagnostics and abort. malloc returning NULL invites missing checks; use a ckalloc that aborts. Likewise for out-of-memory exceptions, I/O errors and inconsistent data structures. It depends on the application; a replicated storage system must recover.

10.9 Design special cases out of existence

- Special cases make code “riddled with if statements”. Students kept a state variable for whether a selection exists; instead, “the selection always exists”, empty when invisible, and copy and delete work without checks. “No selection” is how the user thinks, not what the implementation must represent (“different layer, different abstraction”).

10.10 Taking it too far

- It only works if the information is not needed outside the module. A network module that masked every exception left applications unable to know if messages were lost. “When something is important, it must be exposed.”

10.11 Conclusion

- Redefine semantics to eliminate error conditions; otherwise mask at a low level or aggregate handlers.

How this chapter builds the author’s case

Applies chapters 5 and 8 (hiding; pulling complexity down) to exceptions, and chapter 6 (general-purpose over special-purpose) to aggregation. The selection example repeats the editor case from chapters 6 to 9. It also pairs with chapter 4’s deep-module test: fewer exceptions means a simpler interface.

CHAPTER 11

Design it Twice

Place in the argument — why this chapter, why this title

After nine chapters of design ideas, chapter 11 is about the process for applying them. The title is the rule. It has no sub-headings: one argument, shown on the text-class example from earlier chapters, then why smart people resist it and what it gives you.

- “It’s unlikely that your first thoughts about how to structure a module or system will produce the best design.” Consider several options for each major decision.
- For the text class: line-oriented, character-oriented and string (range) oriented interfaces. Sketch only the main methods. “Try to pick approaches that are radically different from each other”; even if sure of one approach, consider a second.
- List pros and cons. The key criterion is “ease of use for higher level software”, then simplicity, generality, and efficiency. Combine alternatives if that is better. If none is good, the problems drive a new design: both line and character designs needed extra text manipulation in callers, “a red flag”, which leads to a range-oriented API.
- Apply it to implementation (linked list of lines, fixed-size blocks, gap buffer) and to higher levels. It costs little: an hour or two for a class against days or weeks of implementation.
- Smart people often resist, having learned that a first idea earns a good grade: “no-one is good enough to get it right with their first try.” “It isn’t that you aren’t smart; it’s that the problems are really hard!”
- It also builds skill: comparing approaches teaches what makes designs better or worse.

How this chapter builds the author’s case

Follows the strategic investment of chapter 3 with a concrete practice. It reuses chapter 6’s text-class alternatives, ending with the range-oriented design chapters 6 to 8 favour. Chapters 12 to 16 turn to comments, another design tool.

CHAPTER 12

Why Write Comments? The Four Excuses

Place in the argument — why this chapter, why this title

Chapters 4 to 11 covered structure; chapter 12 opens the second theme, documentation, and begins with why. The title promises the four excuses, and the sections answer them in order (12.1-12.4) before stating the benefits (12.5). Comments are placed within design, not apart from it.

Chapter opening — untitled section before the first sub-heading in the book

- "Without comments, you can't hide complexity", and writing comments "if done correctly, will actually improve a system's design." Yet much production code has none, and where they exist they are often mediocre. The chapters aim to convince you that good comments matter, aren't hard to write, and "can actually be fun."

12.1 Good code is self-documenting

- "A delicious myth, like a rumor that ice cream is good for your health." Only signatures can be specified in code; behaviour, meaning of results, rationale and conditions for calling cannot.
- Reading the code instead pushes toward many short, shallow methods, and does not scale.
- "If users must read the code of a method in order to use it, then there is no abstraction." A declaration alone cannot say whether end is included or what happens if start > end. "If you want to use abstractions to hide complexity, comments are essential."

12.2 I don't have time to write comments

- If documentation is de-prioritized "you'll end up with no documentation." Use the investment mindset: typing code is no more than about 10% of development time, so comments add at most about 10%, and the benefits quickly offset it. Comments on abstractions are written in design and "pay for themselves immediately."

12.3 Comments get out of date and become misleading

- Updating needs large changes only when code changes are large. Avoid duplicated documentation and keep it close to the code; code reviews catch stale comments.

12.4 All the comments I have seen are worthless

- "The one with the most merit", but "writing solid documentation is not hard, once you know how."

12.5 Benefits of well-written comments

- Comments capture "information that was in the mind of the designer but couldn't be represented in the code." Without it, later developers rederive or guess, and even you forget after a few weeks.
- They reduce cognitive load and unknown unknowns, clarify dependencies, and fill gaps to eliminate obscurity.

How this chapter builds the author's case

Ties back to chapter 2 (the three symptoms, dependencies and obscurity), chapter 4 (abstractions and informal interfaces) and chapter 3 (investment). Chapter 13 says how to write comments, 14 covers names, 15 comments as a design tool, 16 maintaining them.

CHAPTER 13

Comments Should Describe Things that Aren't Obvious from the Code

Place in the argument — why this chapter, why this title

Chapter 12 argued why to write comments; chapter 13 says what goes in them, and the title is the one guiding principle. The sections go from conventions (13.1), to what to avoid (13.2), to the two directions in which a comment can differ from the code: lower-level precision (13.3) and higher-level intuition (13.4), then the three places comments matter most: interfaces (13.5), implementations (13.6), cross-module decisions (13.7).

Chapter opening — untitled section before the first sub-heading in the book

- "Statements in a programming language can't capture all of the important information that was in the mind of the developer." The guiding principle: "comments should describe things that aren't obvious from the code": range boundaries, why code exists, rules like "always invoke a before b."
- Abstractions are a main reason: developers "should be able to understand the abstraction provided by a module without reading any code other than its externally visible declarations." Good comments work at a different level of detail than the code, lower or higher.

13.1 Pick conventions

- Follow tool conventions (Javadoc, Doxygen, godoc). Conventions ensure consistency and ensure you actually write comments.
- Four categories: **interface** comment (before a class or method), **data structure member** comment, **implementation** comment (inside a method), **cross-module** comment. The first two matter most; "every class should have an interface comment, every class variable should have a comment, and every method should have an interface comment."

13.2 Don't repeat the code

- Test: "could someone who has never seen the code write the comment just by looking at the code next to the comment? If the answer is yes", the comment adds nothing. Comments with the same words as the name (getNormalizedResourceNames: "Obtain a normalized resource name from REQ") are worthless. **Red flag: comment repeats code.**
- Use different words that add meaning: what padding is, its units (pixels) and that it applies to both sides.

13.3 Lower-level comments add precision

- Comments at the same level as the code repeat it; lower-level comments add precision. For declarations, say units, inclusive or exclusive bounds, what null means, who frees a resource, and invariants. "The code" means the code next to the comment, not "all of the code in the application."
- Vague comments for a TreeMap of line widths became precise: keys are lengths in characters, values are counts, and a missing entry means no such lines.
- "Think nouns, not verbs": describe what a variable represents, not how it is toggled.

13.4 Higher-level comments enhance intuition

- A too-detailed comment on a loop over RPCs repeated the code and missed the purpose; the better one says "append to an existing RPC." Ask: "What is this code trying to do? What is the simplest thing you can say that explains everything in the code?"
- Great designers "can also step back from the details." Comments of the form "how we get here", explaining why code runs, are very useful.

13.5 Interface documentation

- "The only way to describe an abstraction is with comments." Separate interface comments from implementation comments: "If interface comments must also describe the implementation, then the class or method is shallow."
- A method's interface comment gives behaviour as callers see it, precise arguments and return value, side effects, exceptions and preconditions.
- IndexLookup example: the original class comment described RPC names and private parameters. The short version says what an instance represents and how to use getNext; it omits server crashes

because they are invisible to users. **Red flag: implementation documentation contaminates interface.** The isReady comment likewise needed rewriting to say what "ready" means.

13.6 Implementation comments: what and why, not how

- Short methods need none. Longer ones need a comment before each major block saying what it does, and before complex loops. Also explain why for tricky code, referring to bug-tracker entries rather than repeating them. Comment a local variable only if used over a large span.

13.7 Cross-module design decisions

- Such decisions "account for many bugs", and the challenge is finding where developers will see the note. If there is a natural place (the Status enum listing the seven other places to update), put it there. Otherwise keep a central designNotes file with a short pointer comment in each related piece of code; one copy, but far from the code.

13.8 Conclusion

- "Obvious" means to someone reading the code for the first time. "If a reader thinks it's not obvious, then it's not obvious."

13.9 Answers to questions from Section 13.5

- A developer needs the comparison function (yes) and possibly whether requests are concurrent; not the message format, the index data structure, or crash handling (crashes are invisible to users).

> **How this chapter builds the author's case**
> Builds on chapter 12 and chapter 4's informal interface and false abstraction; the interface-comment test restates depth (chapter 4) and information hiding (chapter 5). The definition-out-of-existence note (Buffer::copy) cites chapter 10. Chapter 14 turns to names, 15 to when to write comments, 16 to maintaining them.

CHAPTER 14

Choosing Names

Place in the argument — why this chapter, why this title

Chapter 13 handled comments; chapter 14 handles names, the other main way code carries meaning. The title is the topic. It follows chapter 13 because "good names are a form of documentation" that reduce the need for other documentation. The sections go: a cautionary bug (14.1), the aim (14.2), the two properties (14.3 precise, 14.4 consistent), a dissenting view (14.5).

Chapter opening — untitled section before the first sub-heading in the book

- Naming is "one of the most underrated aspects of software design." It is a case of the principle that complexity is incremental: a mediocre name matters little, but "software systems have thousands of variables."

14.1 Example: bad names cause bugs

- The hardest bug the author fixed, in the Sprite operating system, took six months: files occasionally lost a block to zeroes. The variable block meant a physical disk block in some places and a logical file block in others, and one was used where the other was needed. Readers "reflexively assumed" it was a physical block number; fileBlock and diskBlock would have prevented it.
- Don't settle for names "reasonably close": "precise, unambiguous, and intuitive."

14.2 Create an image

- "The goal is to create an image in the mind of the reader." Ask whether someone seeing the name in isolation could guess what it refers to. Limit to two or three words. Names are abstractions that focus on what is most important.

14.3 Names should be precise

- Too vague: getCount (count of what? getActiveIndexlets or numIndexlets); x and y (charIndex and lineIndex); blinkStatus (cursorVisible; "names of boolean variables should always be predicates"); VOTED_FOR_SENTINEL_VALUE (NOT_YET_VOTED); result in a method with no return value. **Red flag: vague name.**
- Exceptions: i and j are fine in short loops. Names can also be too specific: a selection argument that is any range should be range.
- **Red flag: hard to pick name**: a hint "that the underlying object may not have a clean design"; consider alternative factorings, since naming "can improve your design by identifying weaknesses."

14.4 Use names consistently

- Pick one name for each common purpose and use it everywhere. Three requirements: always use the common name for the purpose, never use it for anything else, and make the purpose narrow enough that all variables with the name behave the same (the block bug broke the third). Use prefixes for variants (srcFileBlock, dstFileBlock); always i in outer loops and j in nested ones.

14.5 A different opinion: Go style guide

- Go developers favour very short names: "long names obscure what the code does." The author finds the longer version no harder to read and thinks reused short names (ch for character or channel) cause confusion, but: "readability must be determined by readers, not writers."
- Agreement: "The greater the distance between a name's declaration and its uses, the longer the name should be."

14.6 Conclusion

- Good names make code obvious: the first guess about a variable "will be correct." This is the investment mindset; skill at naming becomes almost free with experience.

How this chapter builds the author's case

Follows chapter 13's obscurity theme (chapter 2) for names; the vague-name and hard-to-pick red flags echo the chapter 4 and 5 tests for depth. Chapter 17 (consistency) and chapter 18 (obvious code) extend the naming points.

CHAPTER 15

Write The Comments First (Use Comments As Part Of The Design Process)

Place in the argument — why this chapter, why this title

Chapters 12 to 14 said why and what to comment; chapter 15 says when, and answers the objection that comments should wait until code is stable. The title and subtitle give the practice and its purpose: comments are part of design, not a record made afterwards. The sections run from why delay fails (15.1) to the method and three benefits (15.2-15.4) and the cost objection (15.5).

Chapter opening — untitled section before the first sub-heading in the book

- Putting off documentation "is one of the surest ways to produce poor quality documentation." Writing comments first "makes documentation part of the design process", giving better documentation, better designs, and more enjoyment.

15.1 Delayed comments are bad comments

- Developers say code is still changing, but also see comments as drudge work. Delaying often means never; by the time code stabilizes, there is a lot of it. If written late, you have "checked out mentally", memories are fuzzy, and comments "repeat the code" and miss the important things.

15.2 Write the comments first

- Method: write the class interface comment; then interface comments and signatures of the main public methods with empty bodies; iterate; then instance variable comments; then bodies. New methods get their interface comment before the body. "When the code is done, the comments are also done."
- Benefit one: better comments, since design issues are fresh and you focus on the abstraction before the implementation.

15.3 Comments are a design tool

- The most important benefit: comments "provide the only way to fully capture abstractions", so writing them early lets you review the abstraction before implementing it.
- "Comments serve as a canary in the coal mine of complexity": a long comment signals a poor abstraction; if the interface comment must describe the implementation, the method is shallow. **Red flag: hard to describe.** This only works if comments are complete and clear.

15.4 Early comments are fun comments

- The early design phase is the most enjoyable part; "finding simple comments is a source of pride." Writing comments is fun when your goal is a great design, which is strategic programming.

15.5 Are early comments expensive?

- Typing code and comments is at most about 10% of development time, comments perhaps 5%, so delaying saves little. Stable abstractions may even save coding time: "it might be faster overall to write the comments first."

15.6 Conclusion

- Try it long enough to get used to it, then judge quality, design and enjoyment.

How this chapter builds the author's case

Combines chapter 3's strategic investment, chapter 4's depth test and chapter 12's cost estimate (10%) with chapter 13's interface comments. It makes comments a design tool before chapter 16 deals with maintaining them and chapter 18 with obvious code.

CHAPTER 16

Modifying Existing Code

Place in the argument — why this chapter, why this title

The earlier chapters dealt with initial design and writing comments; chapter 16 asks how not to undo that work as the system changes. The design of a mature system "is determined more by changes made during the system's evolution than by any initial conception." The first section applies the strategic mindset to changes, and 16.2-16.6 give five ways to keep comments accurate.

Chapter opening — untitled section before the first sub-heading in the book

- This chapter "discusses how to keep complexity from creeping in as the system evolves."

16.1 Stay strategic

- Developers ask "what is the smallest possible change I can make that does what I need?" Each minimal change adds special cases or dependencies, so the design gets worse step by step.
- Aim that after each change the system "will have the structure it would have had if you had designed it from the start with that change in mind." If the design is no longer the best, refactor. "If you're not making the design better, you are probably making it worse."
- Commercial reality (a three-month refactoring against a two-hour quick fix) may force compromise; ask "Is this the best I can possibly do to create a clean system design, given my current constraints?" Plan a small fraction of effort for cleanup.

16.2 Maintaining comments: keep the comments near the code

- Inaccurate comments make readers distrust all comments. "The farther a comment is from its associated code, the less likely it is that it will be updated properly": put interface comments in the code file beside the body, not in a header. Push implementation comments to the narrowest scope, with an optional overview of the phases at the top. "The farther a comment is from the code it describes, the more abstract it should be."

16.3 Comments belong in the code, not the commit log

- Developers rarely scan the log. If information will be needed later (a subtle problem that motivated a change), document it in the code, or someone may undo the change and "re-created a bug."

16.4 Maintaining comments: avoid duplication

- "Document each design decision exactly once", in the most obvious single place (or a designNotes file), with short pointers elsewhere. A broken pointer is self-evident, while a stale copy is not.
- Don't redocument one module's decisions in another, and refer to external documentation (a URL for HTTP, the user manual) instead of repeating it.

16.5 Maintaining comments: check the diffs

- Scan all changes before each commit to check the documentation; this also catches leftover debugging code and TODO items.

16.6 Higher-level comments are easier to maintain

- Comments more abstract than the code are unaffected by minor changes. "The comments that are most useful ... are also easiest to maintain."

How this chapter builds the author's case

Carries chapter 3's strategic investment into maintenance. The keep-it-near-the-code and no-duplication rules answer chapter 12's excuse that comments go out of date (12.3), and the designNotes file comes from chapter 13.7. Chapters 17 and 18 return to keeping code consistent and obvious.

CHAPTER 17

Consistency

Place in the argument — why this chapter, why this title

Chapter 17 generalizes the naming point of chapter 14 (consistency as one of two properties of good names) to the whole system, as another tool for making behaviour obvious. The sections give examples (17.1), how to establish it (17.2), its limit (17.3) and its price (17.4).

Chapter opening — untitled section before the first sub-heading in the book

- "Similar things are done in similar ways, and dissimilar things are done in different ways." Consistency "creates cognitive leverage": what you learn in one place applies elsewhere. It also reduces mistakes, because assumptions from familiar-looking situations are safe.

17.1 Examples of consistency

- **Names** (chapter 14); **coding style** (style guides); **interfaces** with multiple implementations; **design patterns** (more in chapter 19); **invariants**, properties that are always true, such as every line ending in a newline, which reduce special cases.

17.2 Ensuring consistency

- **Document** conventions where developers will see them; local ones such as invariants in the code. **Enforce** with a tool that blocks commits: a pre-commit script rejected carriage returns after Unix and Windows line endings caused whole-file diffs. Nit-picky code reviews educate the team.
- **"When in Rome ..."**: follow the conventions you find in the code (declaration order, camel case or snake case).
- **Don't change existing conventions**: "The value of consistency over inconsistency is almost always greater than the value of one approach over another." Change only if you have significant new information and the improvement is worth updating all old uses.

17.3 Taking it too far

- Dissimilar things must be done differently. Forcing them into one approach (same variable name for different things, a design pattern that doesn't fit) creates confusion. Consistency helps only if "if it looks like an x, it really is an x."

17.4 Conclusion

- Another investment (conventions, checkers, mimicry, reviews); the return is more obvious code, faster understanding and fewer bugs.

How this chapter builds the author's case

Extends chapter 14's consistent names and chapter 2's obscurity to the system as a whole. It prepares chapter 18 (obvious code) and refers ahead to chapter 19 on design patterns.

CHAPTER 18

Code Should be Obvious

Place in the argument — why this chapter, why this title

Chapter 2 named obscurity as one of the two causes of complexity and obviousness as the goal; chapter 18 comes after the chapters on comments, names and consistency, which are its tools. The title is the goal. The sections list what makes code more obvious (18.1), what makes it less (18.2), and restate it as a question of information (18.3).

Chapter opening — untitled section before the first sub-heading in the book

- Code is obvious when "someone can read the code quickly, without much thought, and their first guesses about the behavior or meaning of the code will be correct." It needs fewer comments.
- "Obvious" is in the mind of the reader, so code reviews are the best test: "If someone reading your code says it's not obvious, then it's not obvious."

18.1 Things that make code more obvious

- Two techniques already covered: good names (chapter 14) and consistency (chapter 17).
- **Judicious use of white space**: separating parameter documentation, blank lines before commented blocks, and spacing within statements.
- **Comments**, when nonobvious code is unavoidable: put yourself in the reader's position.

18.2 Things that make code less obvious

- **Event-driven programming**: handlers are invoked indirectly, so it is hard to follow control flow; compensate with interface comments saying when each handler is invoked. **Red flag: nonobvious code.**
- **Generic containers** (Pair): the elements get meaningless names such as getKey(); define a specific class. "Software should be designed for ease of reading, not ease of writing."
- **Different types for declaration and allocation** (declaring a List but allocating an ArrayList): match them.
- **Code that violates reader expectations**: a main that returns while RaftClient threads keep running should carry a comment.

18.3 Conclusion

- Obvious code means readers have the information they need. Three ways: reduce the information needed (abstraction, fewer special cases), use information readers already have (conventions, expectations), or present it in the code (good names, strategic comments).

How this chapter builds the author's case

Applies chapter 2's goal "obvious", combining chapters 13 to 17 (comments, names, consistency). Chapter 19 tests popular practices against these same standards.

CHAPTER 19

Software Trends

Place in the argument — why this chapter, why this title

After the principles, chapter 19 uses them as a test: "for each trend, I will describe how that trend relates to the principles in this book and use the principles to evaluate whether that trend provides leverage against software complexity." The sections take popular ideas in turn, and the conclusion states the lesson as a method.

Chapter opening — untitled section before the first sub-heading in the book

- The chapter considers trends "that have become popular in software development over the last few decades" and evaluates each against complexity.

19.1 Object-oriented programming and inheritance

- Private methods and variables support information hiding. **Interface inheritance** gives leverage: "the more different implementations there are of an interface, the deeper the interface becomes."
- **Implementation inheritance** reduces duplication and change amplification, but creates dependencies and information leakage between parent and subclasses; in the worst case programmers need "complete knowledge of the entire class hierarchy." Use with caution: prefer composition (helper classes), or separate parent state from subclass state.
- OOP "do[es] not, by themselves, guarantee good design": shallow classes or exposed state still mean high complexity.

19.2 Agile development

- Mostly about process, but its incremental, iterative development matches the book's view that you cannot visualize a complex design at the outset.
- Risk: it "can lead to tactical programming" (focus on features; put off general-purpose mechanisms), which argues "against an investment approach." Instead: "the increments of development should be abstractions, not features." Design an abstraction cleanly when needed, and make it somewhat general-purpose.

19.3 Unit tests

- Tests facilitate refactoring; without a suite, developers avoid structural change and complexity accumulates. Example: Tcl's byte-code compiler, where the unit tests meant only one bug turned up after the alpha release.

19.4 Test-driven development

- The author is "not a fan": it "focuses attention on getting specific features working, rather than finding the best design. This is tactical programming pure and simple." Design abstractions all at once. An exception: write the failing test first when fixing a bug.

19.5 Design patterns

- Mostly good, since patterns solve common problems, but "the greatest risk with design patterns is over-application": do not force a problem into a pattern. More patterns are not better.

19.6 Getters and setters

- Better not to expose instance variables at all: it violates information hiding, and getters and setters are shallow, one-line methods. Their popularity shows the risk of assuming a pattern is good.

19.7 Conclusion

- Challenge any new paradigm "from the standpoint of complexity"; some "make complexity worse, not better."

How this chapter builds the author's case

Tests the book's ideas against practice: depth and information hiding (4, 5) against inheritance and getters; strategic investment (3) against agile and test-driven development; consistency and patterns (17). It sets up chapter 20, which asks whether performance goals conflict with simple design.

CHAPTER 20

Designing for Performance

Place in the argument — why this chapter, why this title

Every chapter so far aimed at simplicity; chapter 20 meets the objection "what if you are working on a system that needs to be fast?" The title is that question. The answer is that simplicity "usually makes systems faster." The sections: how much to worry (20.1), measure first (20.2), the redesign method (20.3), a worked example (20.4).

Chapter opening — untitled section before the first sub-heading in the book

- The chapter shows "how to achieve high performance without sacrificing clean design. The most important idea is still simplicity."

20.1 How to think about performance

- Optimizing every statement adds complexity and often doesn't help; ignoring performance risks "death by a thousand cuts", a system 5-10x too slow with no single fix.
- The middle path: choose designs that are "naturally efficient" yet clean. Learn which operations are expensive (network round trips, disk and flash I/O, dynamic allocation, cache misses) with micro-benchmarks. Prefer cheap operations when equally simple, e.g. a hash table (often 5-10x faster) unless you need ordering; structures stored in an array, not pointers to them.
- If speed adds complexity: acceptable if small and hidden, otherwise start simple and optimize later, unless there is clear evidence. RAMCloud bypassed the kernel for networking for lowest latency, then designed the rest for simplicity.
- "Simpler code tends to run faster": no special-case checks, and "deep classes are more efficient than shallow ones", since layer crossings cost.

20.2 Measure before modifying

- "Programmers' intuitions about performance are unreliable." Measure deeply to find the few places where time goes, and as a baseline. If a change makes no measurable difference, back it out unless it simplified the system.

20.3 Design around the critical path

- First look for a "fundamental" fix (a cache, a better algorithm). If none, redesign around the **critical path**: write the smallest code needed for the common case, ignoring existing structure ("the ideal"), then find a clean design close to it. "In my experience it's almost always possible."
- Remove special cases from the critical path, ideally one test up front, with a separate path for special cases.

20.4 An example: RAMCloud Buffers

- Buffers hold messages as discontiguous chunks (internal or external). The common operation is allocating a small internal chunk by enlarging the last one. The original code checked 6 conditions across three shallow layers with identical signatures, "a red flag."
- The redesign handled the path in one method with one test, using an extraAppendBytes variable. Result: 20% less code (1476 lines against 1886), the 1-byte append dropping from 8.8 ns to 4.75 ns, and easier to read.

20.5 Conclusion

- "Clean design and high performance are compatible." Complicated code is slow because of "extraneous or redundant work"; for the few cases needing it, make the critical paths as simple as possible.

How this chapter builds the author's case

Answers the objection to the whole book: chapters 4 and 7 (deep modules, no shallow layers) and 10 (fewer special cases) also help speed. It applies chapter 3's measured investment to performance and chapter 7's red flag in the Buffer example. Chapter 21 closes the book.

CHAPTER 21

Conclusion

Place in the argument — why this chapter, why this title

Closes the book on its first sentence's theme and weighs the cost of the investment it asks for.

- "This book is about one thing: complexity": root causes (dependencies, obscurity), red flags (information leakage, unneeded error conditions, generic names), general ideas (deep and generic classes, defining errors out of existence, separating interface from implementation documentation), and the investment mindset.
- Cost: extra early work. Reward: "Design is a fascinating puzzle", and investments in good design "will pay off quickly"; it "doesn't really take much longer than quick-and-dirty design, once you know how."
- Good designers spend more time designing, which is fun; poor designers "spend most of their time chasing bugs."

How this chapter builds the author's case

Returns to chapters 1 to 3: complexity, then the strategic mindset.

Parts of the Book Not Summarised

Title Page, Copyright, Contents: Publisher front matter, not the author's argument.

Index: A look-up aid compiled for the book, not authored prose.

Summary of Design Principles: A one-page list at the back that restates principles already covered in the chapters; its entries are pointers back to those chapters, so it is not repeated here.

Summary of Red Flags: Likewise a list at the back (the author says in section 1.1 that "the most important ones are summarized at the back of the book"); each red flag is noted in the chapter where he introduces it.

About the Author: Biographical note.